

Chapter 9

Transaction Management and Concurrency Control

In this chapter, you will learn:

- What a database transaction is and what its properties are
- How database transactions are managed
- What concurrency control is and what role it plays in maintaining the database's integrity
- What locking methods are and how they work
- How database recovery management is used to maintain database integrity

What is a Transaction?

Transaction

A transaction is a collection of actions that make consistent transformations of system states while preserving system consistency.

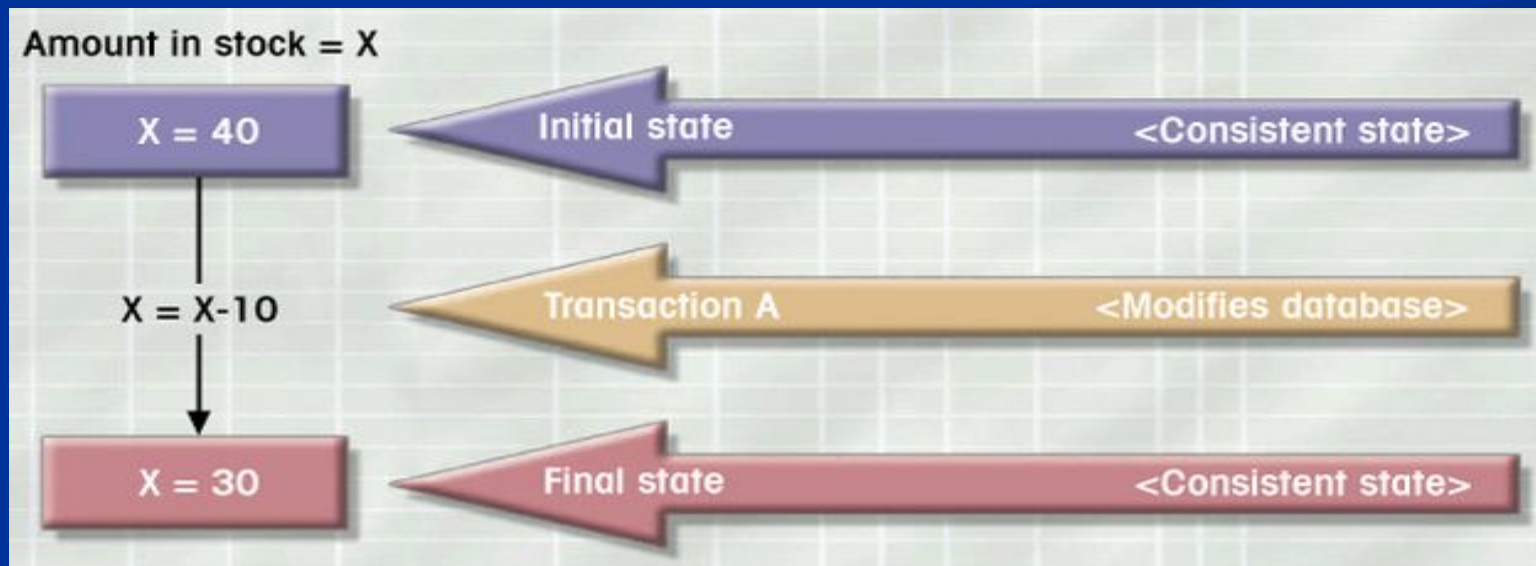
⇒ concurrency transparency

⇒ failure transparency

- Read from or write to a database, create a transaction.
- It can be a single SQL command or series of related commands
- In practical, transactions are formed by two or more database requests
- Database request is the equivalent of a single SQL statement

What is a Transaction?

- Logical unit of work
- Must be either entirely completed or aborted
- No intermediate states are acceptable



What is a Transaction?

- Transaction changes the contents of the database must alter the database from one consistent state to another.
- Consistent database state is one in which all data integrity constraints are satisfied.
- All transactions are controlled and executed by the DBMS to guarantee database integrity.

Transaction Example

Consider an airline reservation example with the relations:

FLIGHT(FNO, DATE, SRC, DEST, STSOLD, CAP)

CUST(CNAME, ADDR, BAL)

FC(FNO, DATE, CNAME, SPECIAL)

Begin_transaction Reservation

begin

input(flight_no, date, customer_name);

 EXEC SQL UPDATE FLIGHT
 SET STSOLD = STSOLD + 1
 WHERE FNO = flight_no AND DATE = date;

 EXEC SQL INSERT
 INTO FC(FNO, DATE, CNAME, SPECIAL);
 VALUES (flight_no, date, customer_name, **null**);

output("reservation completed")

end . {Reservation}

Termination of Transactions

Begin_transaction Reservation

begin

input(flight_no, date, customer_name);

EXEC SQL SELECT STSOLD,CAP

INTO temp1,temp2

FROM FLIGHT

WHERE FNO = flight_no AND DATE = date;

if temp1 = temp2 then

output("no free seats");

Abort

else

EXEC SQL UPDATE FLIGHT

SET STSOLD = STSOLD + 1

WHERE FNO = flight_no AND DATE = date;

EXEC SQL INSERT

INTO FC(FNO, DATE, CNAME, SPECIAL);

VALUES (flight_no, date, customer_name, null);

Commit

output("reservation completed")

endif

end . {Reservation}

Example Transaction

- Register credit sale of 100 units of product X to customer Y for \$500
 - Reduce product X's quantity by 100
 - Adding \$500 to customer Y's account receivable

```
UPDATE PRODUCT
SET PROD_QOH = PROD_QOH - 100
WHERE PROD_CODE = 'X';
UPDATE ACCT_RECEIVABLE
SET ACCT_BALANCE = ACCT_BALANCE + 500
WHERE ACCT_NUM = 'Y';
```


Properties of Transactions

A TOMICITY

⇒ all or nothing

CONSISTENCY

⇒ no violation of integrity constraints

ISOLATION

⇒ concurrent changes invisible È serializable

DURABILITY

⇒ committed updates persist

Properties of Transactions

A TOMICITY

⇒ all or nothing

CONSISTENCY

⇒ no violation of integrity constraints

ISOLATION

⇒ concurrent changes invisible È serializable

DURABILITY

⇒ committed updates persist

Transaction Properties

Atomicity

- Either **all or none** of the transaction's operations are performed.
- Atomicity requires that if a transaction is interrupted by a failure, its partial results must be **undone**.

Consistency

- Internal consistency
 - ⇒ A transaction which executes *alone* against a *consistent* database leaves it in a consistent state.
 - ⇒ Transactions do not violate database integrity constraints.
- Transactions are **correct** programs

Transaction Properties

Isolation

■ Serializability

- ➡ If several transactions are executed concurrently, the results must be the same as if they were executed serially in some order.

■ Incomplete results

- ➡ An incomplete transaction cannot reveal its results to other transactions before its commitment.
- ➡ Necessary to avoid cascading aborts.

Isolation Example

- Consider the following two transactions:

T_1 :	Read(x)	T_2 :	Read(x)
	$x \leftarrow x+1$		$x \leftarrow x+1$
	Write(x)		Write(x)
	Commit		Commit

- Possible execution sequences:

T_1 :	Read(x)	T_1 :	Read(x)
T_1 :	$x \leftarrow x+1$	T_1 :	$x \leftarrow x+1$
T_1 :	Write(x)	T_2 :	Read(x)
T_1 :	Commit	T_1 :	Write(x)
T_2 :	Read(x)	T_2 :	$x \leftarrow x+1$
T_2 :	$x \leftarrow x+1$	T_2 :	Write(x)
T_2 :	Write(x)	T_1 :	Commit
T_2 :	Commit	T_2 :	Commit

Transaction Properties

Durability

- Once a transaction commits, the system must guarantee that the results of its operations will never be lost, in spite of subsequent failures.
- Database recovery

Transaction Structure

■ Flat transaction

- ➡ Consists of a sequence of **primitive** operations embraced between a **begin** and **end** markers.

Begin_transaction Reservation

...

end.

■ Nested transaction

- ➡ The operations of a transaction may themselves be transactions.

Begin_transaction Reservation

...

Begin_transaction Airline

— ...

end. {Airline}

Begin_transaction Hotel

...

end. {Hotel}

end. {Reservation}

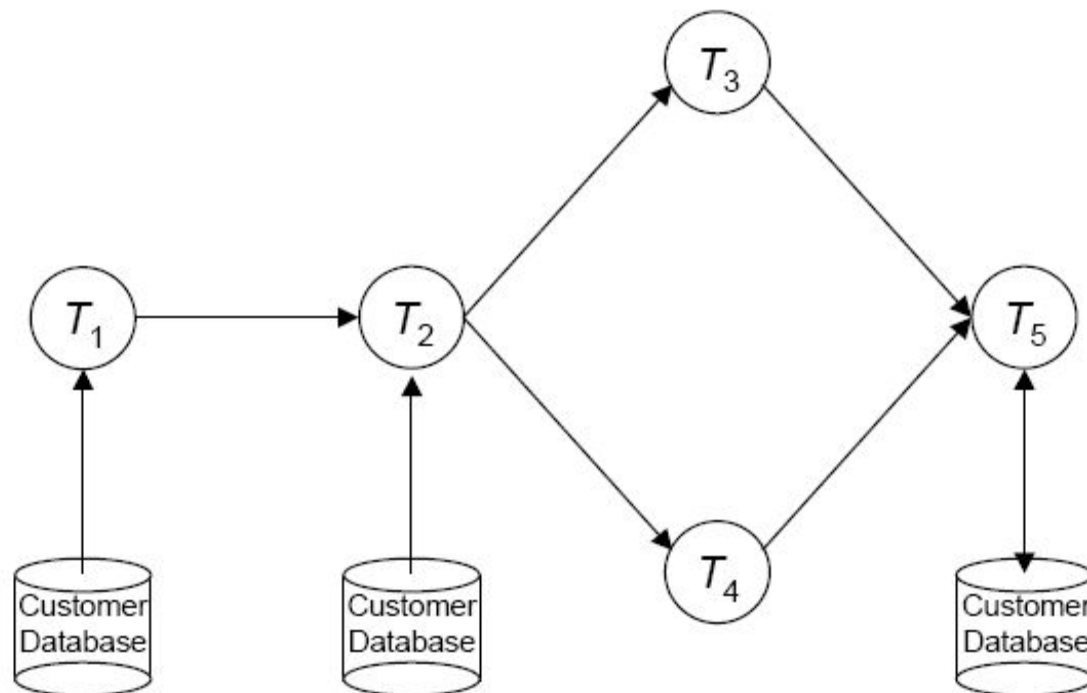
Nested Transactions

- Have the same properties as their parents $\Rightarrow$ may themselves have other nested transactions.
- Introduces concurrency control and recovery concepts to within the transaction.
- Types
 - ▢ Closed nesting
 - ◆ Subtransactions begin *after* their parents and finish *before* them.
 - ◆ Commitment of a subtransaction is conditional upon the commitment of the parent (commitment through the root).
 - ▢ Open nesting
 - ◆ Subtransactions can execute and commit independently.
 - ◆ Compensation may be necessary.

Workflows

- “A collection of tasks organized to accomplish some business process.” [D. Georgakopoulos]

Workflow Example



- T_1 : Customer request obtained
- T_2 : Airline reservation performed
- T_3 : Hotel reservation performed
- T_4 : Auto reservation performed
- T_5 : Bill generated

Logging

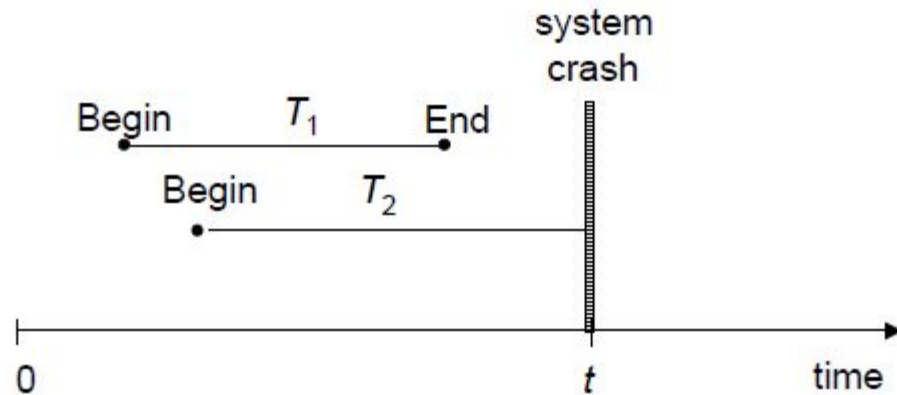
The log contains information used by the recovery process to restore the consistency of a system. This information may include

- ➡ transaction identifier
- ➡ type of operation (action)
- ➡ items accessed by the transaction to perform the action
- ➡ old value (state) of item (**before image**)
- ➡ new value (state) of item (**after image**)

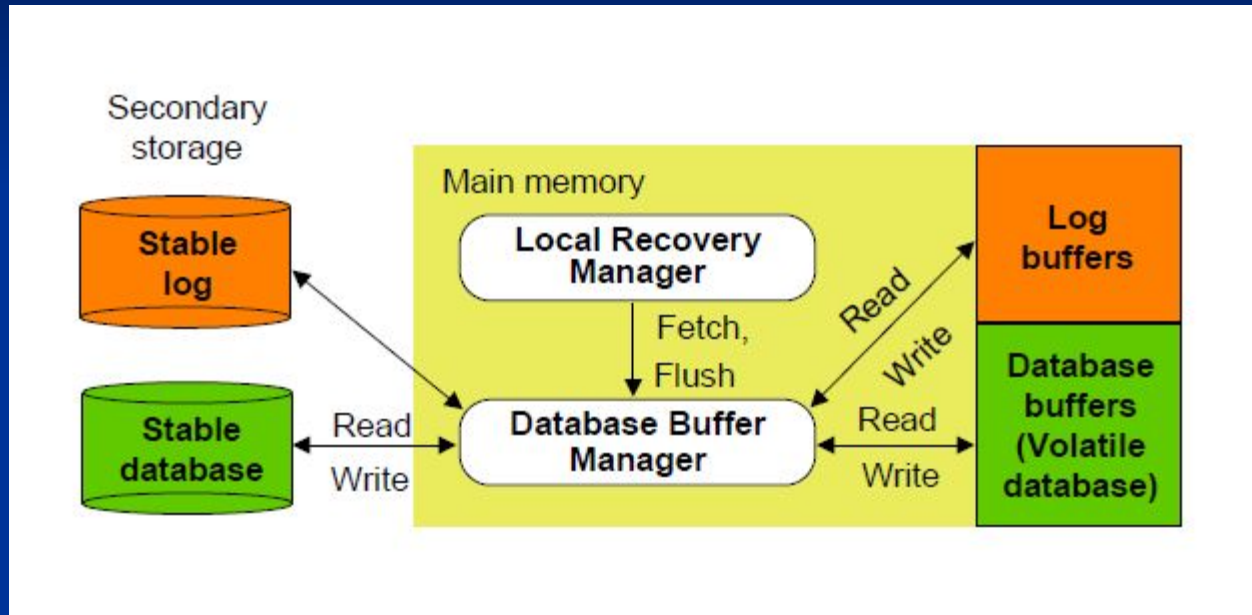
Logging (redo-undo)

Upon recovery:

- ➡ all of T_1 's effects should be reflected in the database (REDO if necessary due to a failure)
- ➡ none of T_2 's effects should be reflected in the database (UNDO if necessary)



Architecture



Fix, Flush, Checkpoint

Media Failures – Full Architecture

